

This document outlines two different projects. An image detection model and a statistics-based estimation model. While different architecturally, the image detection model works with the estimation model to make a better guess.

1. How the two systems fit together

The smoke path starts with a camera frame. A segmentation model paints the visible smoke plume. A separate geolocation step can turn a camera bearing into a rough fire location, and multiple bearings can be triangulated. That position can help an analyst or a larger fusion system, however it is not a mapped burn edge.

The perimeter path starts from a known perimeter or origin, then collects observations such as GOES fire detections, VIIRS hotspots, and occasional sharper imagery or mapped perimeters. The output is a polygon or field representing burned area. Different versions used different rules to turn observations into that shape.

Piece	Input	Output	What has actually been measured
Smoke segmentation	Ground camera image	Smoke mask and score	Held-out mask mAP50; split quality matters
Camera location	Smoke mask, camera coordinates and heading	Bearing or triangulated point	139 m on synthetic geometry; 6.4 km in one live Lucas example
Perimeter estimate	Satellite detections, prior perimeter and context	Burned-area polygon or probability field	LOFO growth metrics on sparse mapped-perimeter pairs
Five-minute loop	Scheduled data polling and state update	Periodic refreshed estimate	Runtime and source plumbing exist; five-minute truth accuracy is not established

2. Camera smoke detection

The camera work moved from fire/smoke boxes toward plume segmentation because smoke is broad and changes shape. A box can say roughly where smoke is; however, a mask gives a more useful outline and can provide valuable information such as plume direction. That direction is still not the ground location of the fire because wind pushes the visible plume away from its source. Prior to this, the model used a box which also included things that were not smoke, confusing the model slightly. In addition, a mask gives a better outline towards the base of the fire which is more representative of the fires footprint rather than the entirety of the smoke outlined by the box.

Data and labeling loop

A dataset of over 3,300 labeled frames in the YOLO smoke set and a review pool of about 3,100 camera images is currently powering the image model. I set up a script to serve as a pipeline allowing me to review images rapidly and relabel them if necessary. The smoke label uses brush-painted polygons. Existing model predictions can seed the label so a reviewer edits a starting mask instead of drawing from scratch. SAM2 was used to convert box annotations to silhouette masks, with a reported 97 percent conversion success rate.

The practical loop was: collect frames, label or correct masks, train, run the model over unreviewed frames, then bring uncertain predictions back for review. The saved scripts include a review server, a brush labeler, SAM2 polygon conversion, inference over the review pool, and retraining. I was able to utilize public camera networks AlertWEST, AlertCalifornia, HPWREN cameras, as well as DOT cameras for states in the Western United States to run the system on those images and label photos. I would then review any detections and, if necessary, relabel them and confirm or deny the labels.

I found that AlertWEST, AlertCalifornia, and HPWREN cameras were the most useful as their resolution was much higher and they publicly listed camera details.



AlertCaliforniacameraframefromSignal PeakLookoutonSeptember15, 2026. The imageshowssmoke, butthephoto by itself isnota segmentationevaluation.

Smoke detection: box versus segmentation

Same camera frame, two different spatial outputs

A Detection box



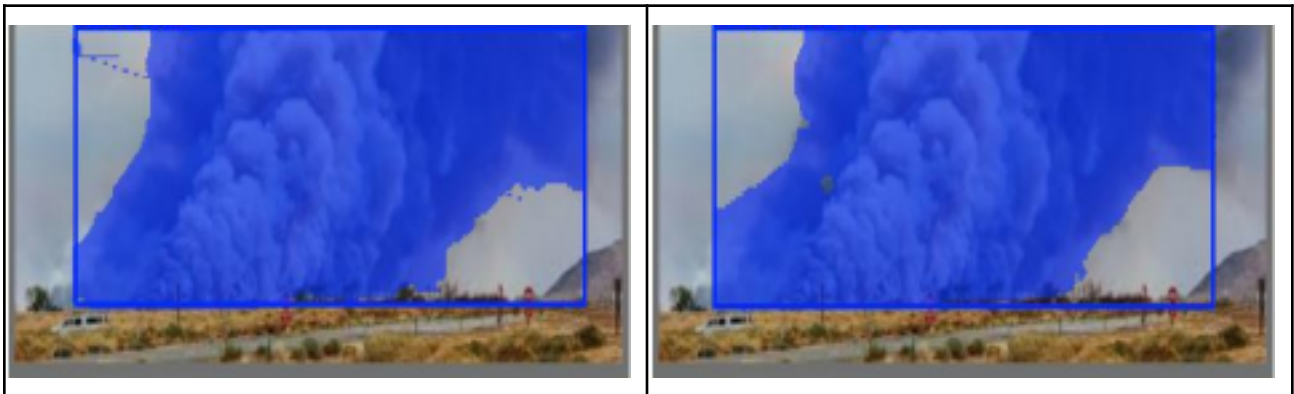
Bounding box

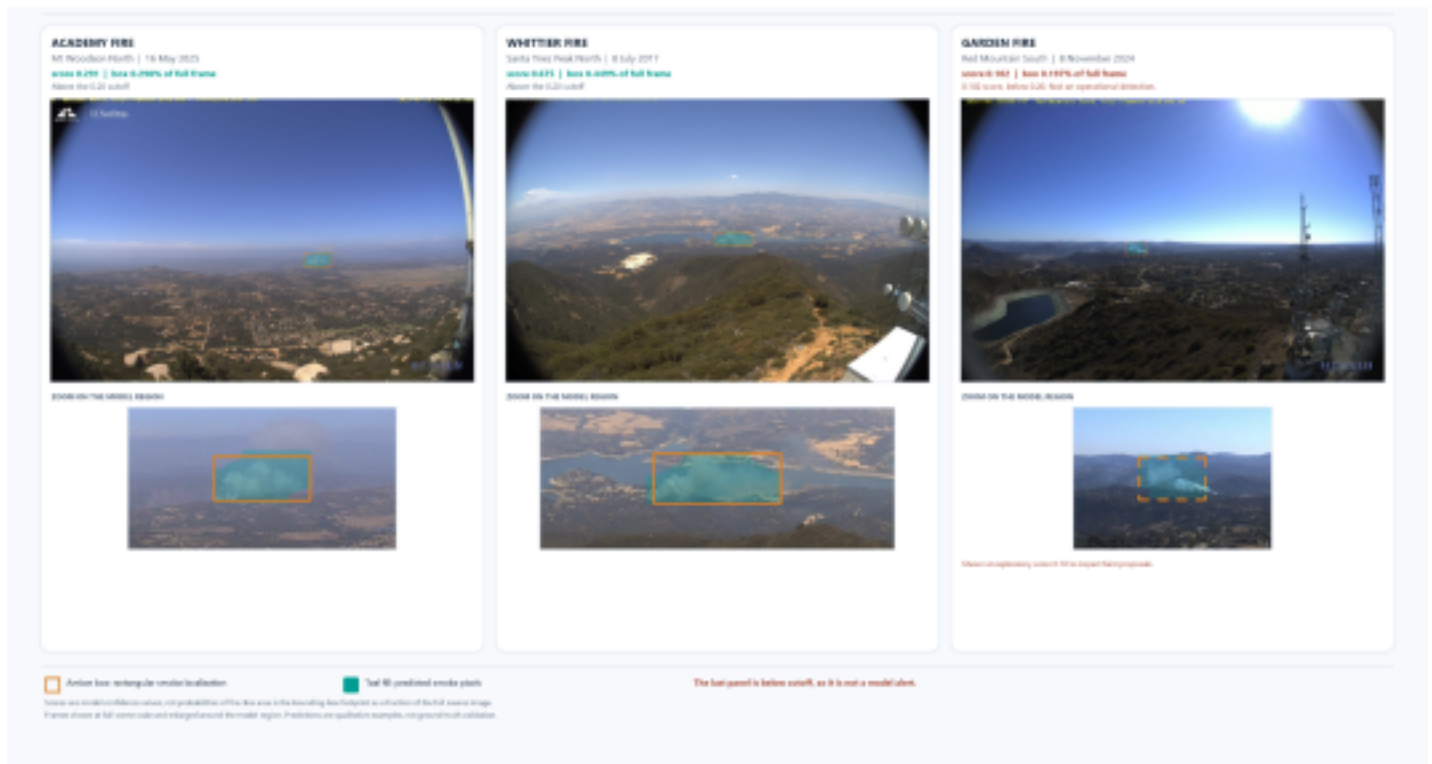
B Segmentation mask



Smoke mask

Source: AlertCalifornia, Signal Peak Lookout, 15 September 2026.





3. Smoke model versions and the split correction

The smoke models have evolved since May; however, the very first models do not exist, and many of the models in June were corrupted, losing their scores.

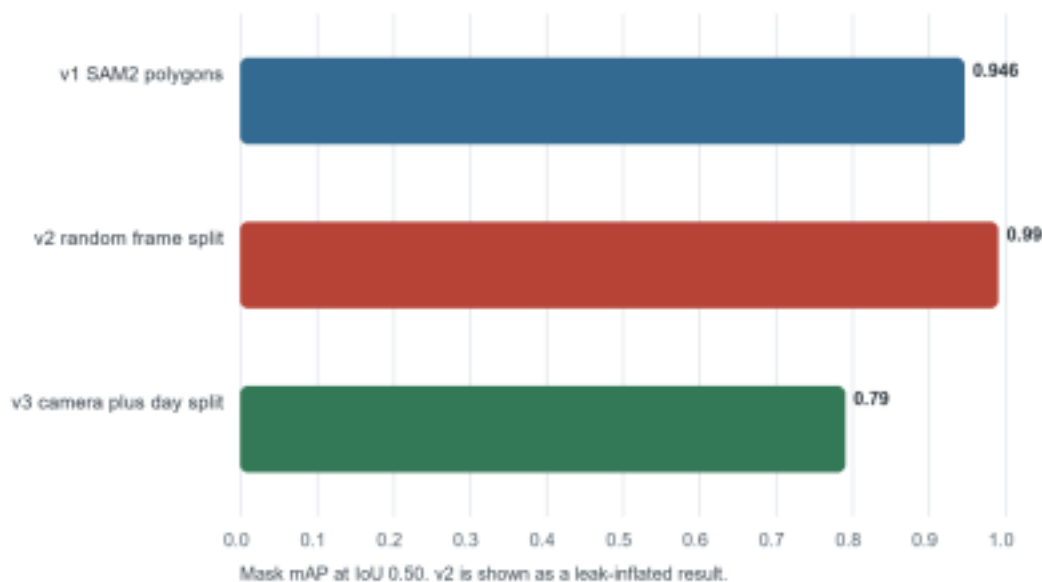
Box detector release	Saved date	Individual score retained?
best_v1_20260628.onnx	June 28	No per-release scorecard found
best_v2_20260628.onnx	June 28	No per-release scorecard found
best_v3_20260629.onnx	June 29	No per-release scorecard found
best_v4_20260629.onnx	June 29	No per-release scorecard found
best_v5_20260629.onnx	June 29	No per-release scorecard found
best_v6_20260630.onnx	June 30	No per-release scorecard found
best_v7_20260729.onnx	July 29	No per-release scorecard found

Version Approach	Mask mAP50 Mask mAP50-95 Read this as
Box detector Original YOLO fire/smoke boxes	Not reported Not reported Useful for finding candidates, but boxes are coarse for diffuse smoke.
Seg v1 SAM2-generated masks, YOLO11-seg	0.946 Not reported Promising first segmentation result. Split details are less complete in the retained summary.
Seg v2 Larger segmentation model, random frame split	0.990 0.609, later 0.773 after masks regenerated Inflated by near-duplicate camera frames appearing on both sides of validation.

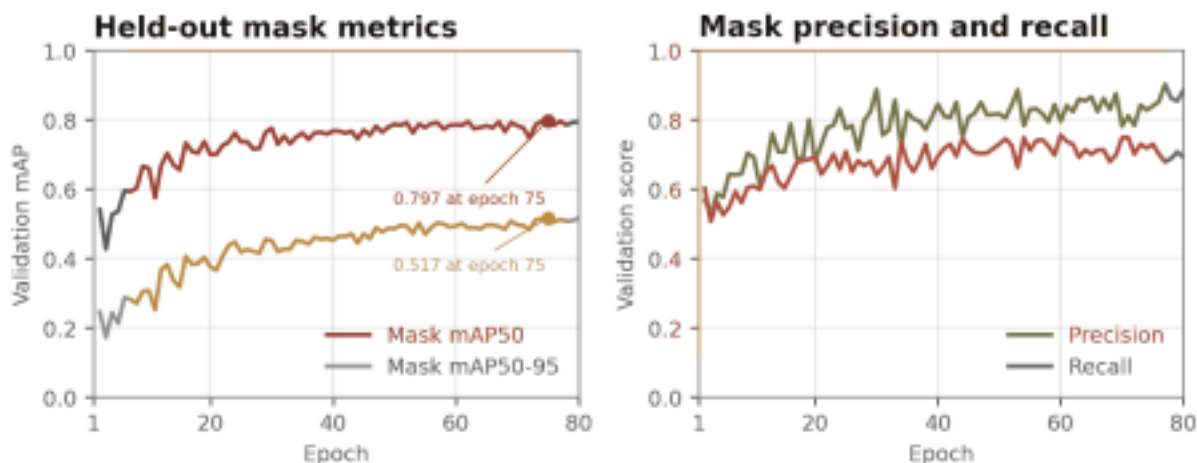
Seg v3 Camera plus day grouped split

About 0.790 About 0.460 Best honest summary in the retained project report.
Whole camera-days stay together.

Smoke mask validation score



YOLO segmentation fine-tuning on grouped camera-day/source splits



Changes

The camera data is correlated in time. A sequence of frames from one camera can look almost identical from one minute to the next. A random frame split put near duplicates in both training and validation, so validation partly tested recognition of scenes the model had already seen. To fix this the camera and day were also taken into grouping. From this, the score fell to about 0.79. That number is less exciting and far more useful as it provides helpful insights into REAL issues.

The v3 segmentation checkpoint was fine-tuned from v2 for 80 epochs at 1024 pixels, a batch size of 8, on the CUDA GPU. Its saved configuration uses mosaic at 0.4, brightness and color changes, and pretrained weights. The earlier v1 model was a 100-epoch YOLO11n-seg run at 640 pixels and batch size 16. The separate other YOLO box retraining script has its own day/night augmentation defaults.

4. Smoke to location system

As a camera observes a direction to a plume, not a fire coordinate. The location code projects the plume base into a compass bearing. With two or more camera bearings, it searches for a consensus intersection using RANSAC. The plume base is used because the top of the plume is carried farther by wind. Plume tilt can also serve as a high-cadence wind-direction clue.

Synthetic triangulation	139 m median error	Known camera geometry and synthetic setup. This is a geometry check, not field accuracy.
Lucas live example	6.4 km location error	A daytime live example; plume drift was the likely limiting bias. One case is not a general result.
Smoke mask	About 0.79 mask mAP50	Camera-day grouped validation, not future-fire field validation.

The useful engineering steps

- Move from rectangles to plume masks so the visible smoke direction can be estimated.
- Use human review to fix labels and recycle model predictions into the next annotation batch.
- Use camera and day groups for validation so adjacent images do not make the score look better than it is.
- Treat a plume bearing as an uncertain direction. Do not draw a fire point at the plume top.
- Keep triangulation error separate from mask quality. A clean mask can still produce a bad location if wind drift or camera geometry is wrong.

Gaps

- Low confidence on smaller smoke plumes
- False identification of water droplets, fog, etc

Next Steps

- Include more examples of small smoke columns
- Label and train using HPWREN dataset
- (hopefully) reach 0.8 IoU on v4

5. Perimeter System

The goal of this system was to generate a perimeter of an active fire using FREE publicly available data. This system is a passion project. It began with GOES-18,19 FRP data points but has expanded to many data sources for increased accuracy.

Project Data Bearing

Source	Useful part Problem for a current boundary
GOES ABI FDCC	about five-minute cadence Pixels are about 2 km; a detected fire is NOT a 2 km filled burn Repeated fire and quality observations at
VIIRS active fire	375 m points that can sharpen shape when a satellite pass occurs Updates every few hours
Mapped perimeters	Much more accurate and more similar to real IR flights flown by aerial resources.]\
Ground cameras	Frequent smoke view and bearing Plume is displaced by wind visible smoke is not a burned-area edge so logical reasoning is necessary

Terrain and fuel	Can constrain plausible spread direction and speed Historical data is constrained
------------------	--

The central design idea was to carry a known perimeter forward and use each new observation as evidence about change. A source that is missing or obscured has to stay unknown.

An additional issue is timing. Archived acquisition timestamps say when a sensor looked. They do not prove when the file reached the system. Historical GOES and VIIRS replay in the retained project notes is therefore marked retrospective acquisition-only wherever receipt time is absent.

Description/Explanation

To train the algorithm/model, I used historical fires starting from March of this year. This is because I found that these fires had the most data easily accessible to me, which would in turn provide the most realistic results, as my system would be working live and would have all the same data accessible. Additionally, there was no shortage of wildfires during this period, especially in the Pacific Northwest (specifically Washington and Oregon), and because strong thunderstorms had started many of these fires, the fires were widespread and burned in many different terrains and fuels. Wildfires in Utah, Colorado, New Mexico, Nevada, and California were also used, with California wildfires being the most “accurate,” with FIRIS intel aircraft mappings more frequently. However, to date (September 2026), no megafires had burned in the state of California, and only 8 fires greater than 10,000 acres had burned (with a few burning into Nevada). As I soon found out, the larger the fire, the more accurate the system would be.

To identify a good testing fire, I looked for fires with extensive perimeter/growth documentation as well as complete GOES FRP (and later other satellite products) data. Once I had identified these fires, I then downloaded all relevant data for each fire before then running a simple training pipeline to test.

The first step of the pipeline was to identify a step between two mapped fire perimeters for one fire; these “steps” were also grouped (explained later) based on several factors. Once a step was identified, the system would pull all FRP data from the time the first perimeter in the set was uploaded to the second perimeter of the step. Data was pulled from 30 minutes before the first perimeter was uploaded to ensure that all relevant FRP points would be included in case of slow delivery times.

After this step, the model/algorithm would ingest this data and produce a result; this result would generate an inferred perimeter for the period of time between step one and step two without looking at step two. That means that it would add onto the existing first perimeter in the step solely using FRP and other data necessary to the model. Once the perimeter was generated, the acreage of the generated perimeter would then be calculated, and using percent error, a percentage would be calculated for how close the generated perimeter was to the real perimeter. However, this only found magnitude, so to combat this, the perimeter’s IoU(Intersection over Union) relative to the real perimeter would be calculated.

Once the set of fires was done, the results of each model run through the different steps of each fire would be averaged out, producing an average as well as a median result which would then be communicated to me.

At first, I relied only on GOES data. I quickly found that GOES fire radiative power (FRP) measurements were not enough on their own to estimate how much of an area had burned. FRP can vary with terrain, vegetation and fuel, weather, and fire intensity. Some readings may also be false positives. As a result, two tiles that were both completely burned could have very different maximum FRP values. One might peak around 500 MW, while another could exceed 10,000 MW. Therefore, a single FRP threshold would not work reliably for every fire.

To address this, I considered comparing each tile’s current FRP with its past FRP from the same fire. When current data show that a fire is active, the next question is which tiles contain active burning. If a tile’s current FRP is similar to values previously recorded in that tile during the same fire, that is evidence that the fire may be burning there. Historical fire perimeters can help check this estimate. When those are unavailable, comparing a tile’s FRP with the values in nearby tiles can still provide useful context.

High-intensity fires create an additional challenge. Their strong FRP readings can affect adjacent tiles, making those tiles appear to be burning when the fire may not have reached them. Looking at how FRP values change between neighboring tiles, along with the direction those values are developing, can help estimate the location and movement of the fire front more

precisely.

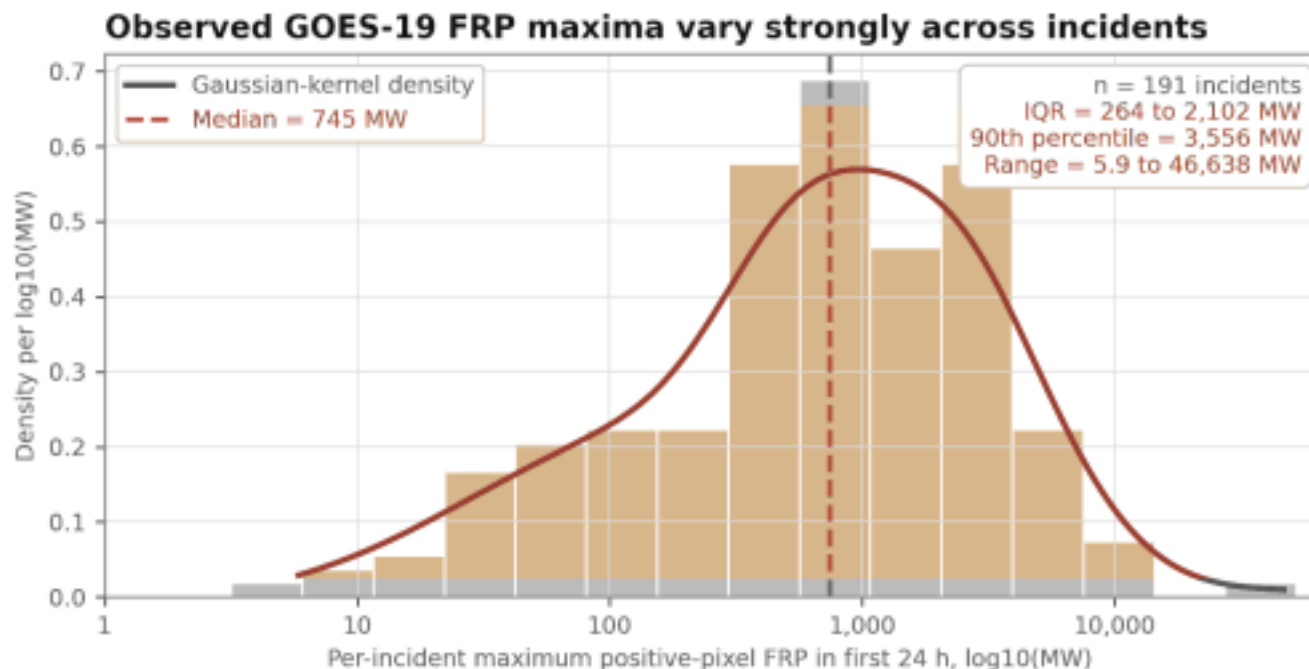
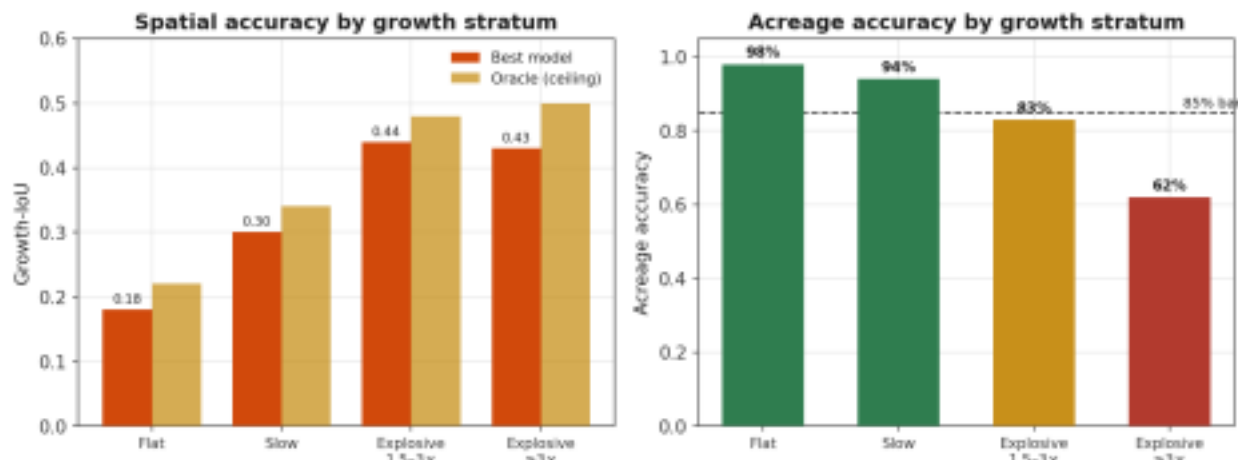


Figure 1. Maximum FRP across incidents.

As shown, different fires can produce drastically differing FRP values. Because of this, it was important not only to implement the adjacent tile system, but to classify fires into different types.

Figure 2. Accuracy across fire growth categories.



Explosive fires are the hard regime; acreage falls from 98% (Flat) to 62% (≥3x); spatial accuracy caps near the oracle.

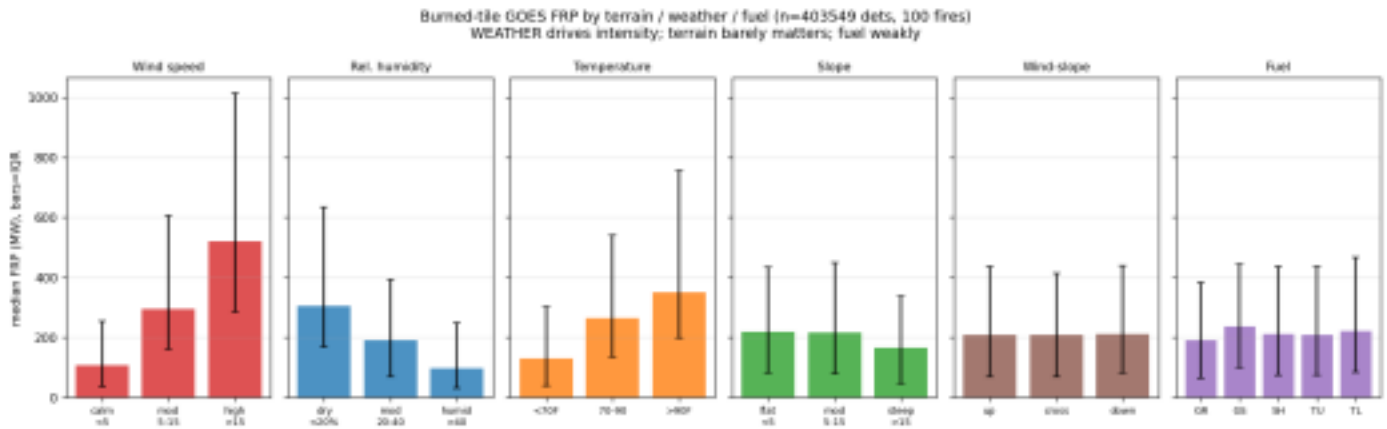
Disclaimer: The data shown here represents an outdated model and represents an issue found in testing

Early in testing, I noticed that some of the model's inaccuracies for certain fires could be grouped into categories. Specifically by rate of growth. The fire was excellent at predicting the acreage of slow-growing and flat fires; however, with fires whose steps exceeded an acreage multiplier (e.g. 1000 acres to 20,000 acres between two perimeters would be a 20x multiplier) of 3x, it would show a significant drop of more than 20% in acreage accuracy compared to steps with an acreage growth multiplier of 1.5-3x. This made me realize that the system would need to be able to handle and identify these scenarios of rapid growth. I identified that typically rapid growth would be marked by extremely high FRP values, and FRP values far from the last mapped perimeter. However, at the time, the detector was throwing away high FRP values and far-away FRP values as misreads. The issue was that when I removed this feature, generated fire perimeters would merge with nearby fires due to lack of a proper gate. Additionally, with this gate removed, lower FRP values would not be seen as "significant" enough to signal a burnt tile, inversely harming the acreage accuracy of fires with lower FRP values.

To combat this, I added a system of continuity where the FRP values of a step must be connected to each other, and tuned the existing, but poorly functioning, system of relative intensity. This made sure that:

1. Distance FRP values that were physically disconnected from the FRP values of a fire were not included
2. FRP values would only be compared relative to the FRP values of similar fires, and/or the same fire to identify fire growth.

Figure 3. FRP across environmental groups.



Additionally, it was necessary for fires to be classified into types (such as explosive, slow, flat, etc.), however, in a real-life scenario, I would not have a second perimeter to compare against. Thus, I ran an experiment to see how much different “fire-boosting” conditions affect FRP values. In this experiment, I found that weather

conditions had the largest effect on fire growth and classification. Fires with specific weather conditions would produce much greater FRP values and grow much larger. I also found that terrain had little to no impact on FRP values, likely because each tile is a 2km by 2km area, meaning that while there would be drainage for the fire to burn up, there would also be valleys for the fire to back down.

Since tiles are 2km x 2km the larger the fire, the easier it is to trace the perimeter meaning larger fires had higher accuracy. Since a 2km x 2km area is only around 1000 acres, it is entirely possible for a fire to burn completely inside of a 2kmx2km area without FRP values ever reaching another tile. In the scenarios where a fire burns “between” tiles, it is nearly impossible to pinpoint the direction of fire spread or exactly how much fire is burning in each tile. Additionally, it is exceedingly difficult to tell what part of the fire is actively burning in smaller fires.

6. How the perimeter model evolved

The perimeter work has two branches. The operational fusion stack combines sources and runs on a five-minute schedule. The research lab compares algorithms against historical mapped perimeters. They overlap in data and ideas, but they are not one model with one validation score.

Generation	What changed Why it was tried	What the notes say
Simulation v1	Minimum-travel-time raster with a 16-neighbor Dijkstra front Get a baseline physical spread surface	Superseded by wavelet physics.
Simulation v2	Utilized wind, gusts, spotting, and diurnal effects Make spread direction and Huygens wavelets with Rothermal Spreadwind, gusts Shape responds to terrain and weather	Kept as a physics reference, but did not win as a direct perimeter estimator.
ML correction v1 to v5	LightGBM correction to rate of spread Correct physics speed from historical observations	Retired. It learned a day/night duty-cycle mismatch between interval-averaged truth and instantaneous ROS.
Fusion v2	Bayesian belief grid with weighted log-odds votes Combine many sources without last-write-wins behavior	Framework was kept. Missing sources must contribute no evidence.
Fusion v3	Adapted new satellite sources such as Sentinel-2 data	Wired into the live stack. The presence of an adapter does not prove it improves held-out perimeter accuracy.

Frontier champion	Use the strongest Dual-GOES detections push the known edge measured detection-driven baseline	Repeated as the operational comparison. Later complete replay kept this unchanged.
Learned field v1 to v3	Use local detection geometry then a selected area is polygonized Per-cell burned probability on a 250m grid	Best reported research family, but not accepted as a five-minute validated product.

Rothermal Physics

Because a stand-alone Rothermal reachability surface did not beat simple distance at separating useful from misleading detections. A separate training run for rate correction learned the typical fraction of a day that fires actively spread, because the labels averaged change over an interval while the simulator represented instantaneous spread. The learned rate correction was switched off. That meant that terrain and fuel remained useful as context features and physical constraints, but physics was no longer treated as the answer by itself.

Priorities

As the biggest inaccuracies were found in fire with explosive growth, the pipeline was changed to identify explosive steps with all available data and then test each new model on these explosive steps. It is important to note that these models are not based on machine learning and are purely data-based, meaning that this testing would not shift the results of the model significantly for smaller fires.

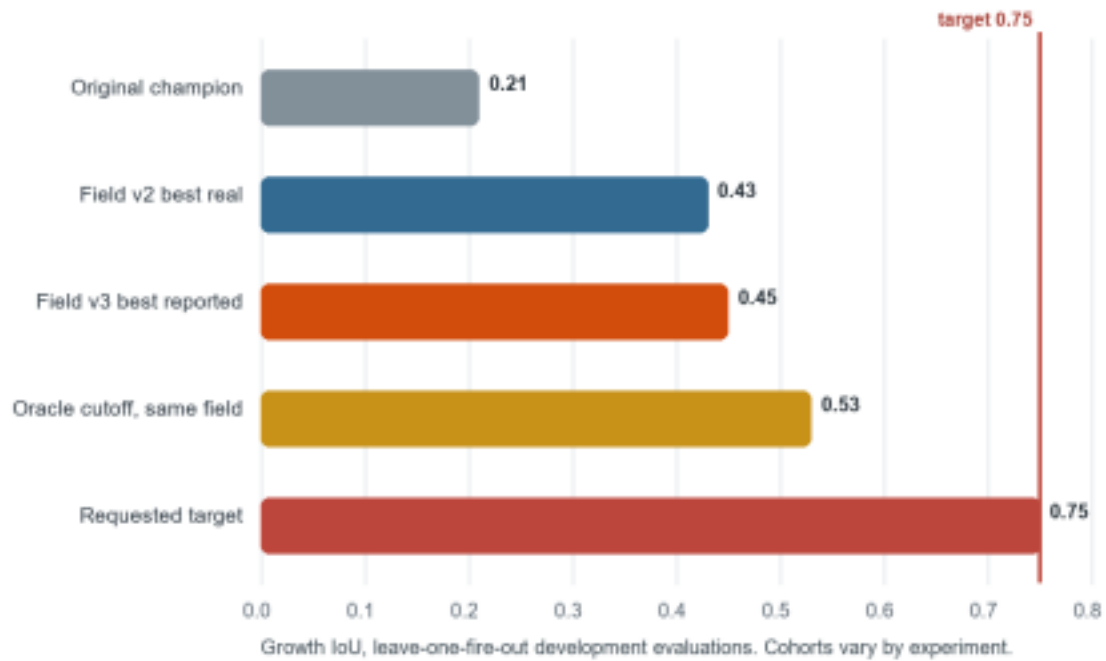
7. Main perimeter result

Growth IoU scores overlap in the new growth outside the known starting perimeter. It does not let persistence earn a high score just by keeping the old area. That makes it more demanding than whole-perimeter overlap.

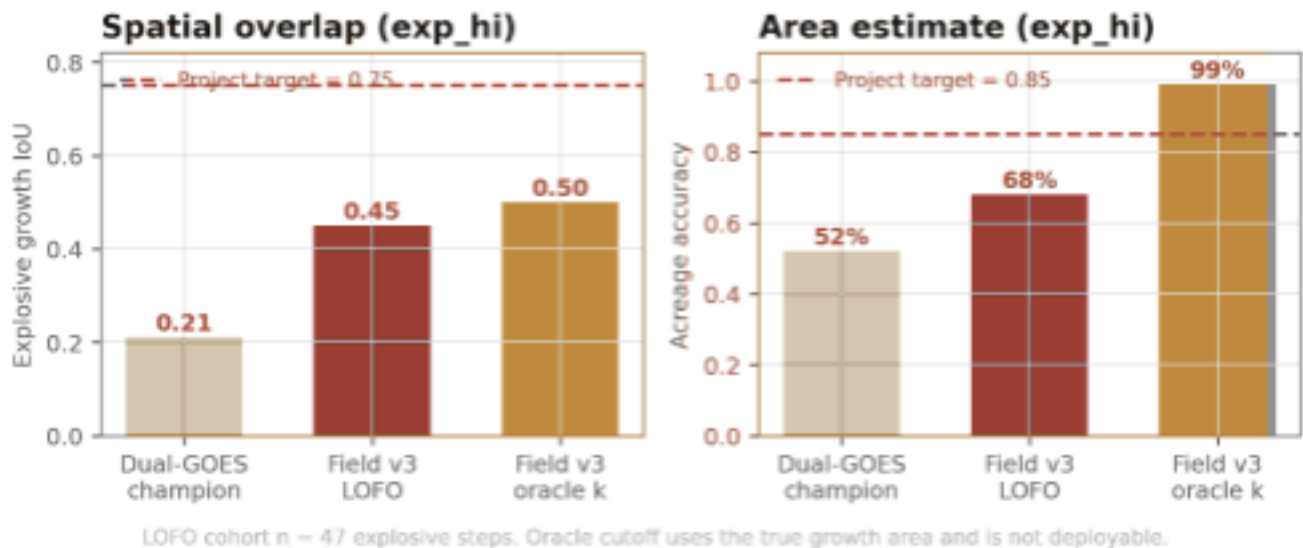
In the wide, data-complete development cohort, the retained results report roughly 0.21 explosive-fire growth IoU for the champion, 0.43 for field v2 with a learned area multiplier, and 0.45 for field v3 at its best reported operating point. An oracle cutoff that knows the true amount of growth reaches about 0.50 on the same reported target stratum. A separate strict-31 figure reports about 0.44 for the best field and 0.606 for the oracle. Those cohort values are shown side by side in the saved presentation figure, but they are not interchangeable.

Figure 4. Perimeter overlap during rapid growth.

Explosive fire growth overlap



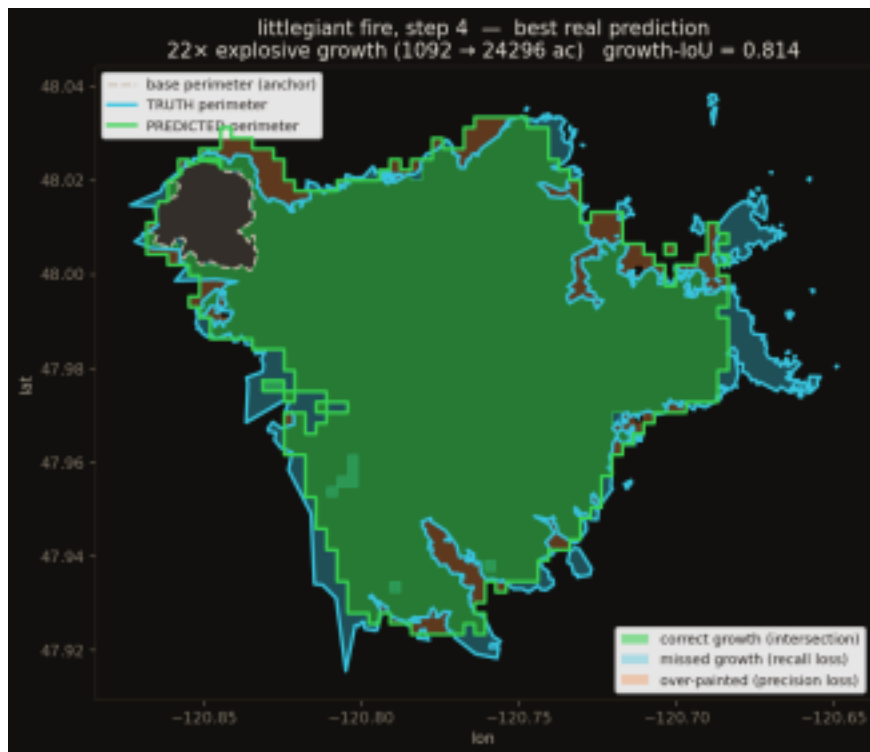
Best retained burned-probability field result on the $\geq 3x$ growth stratum



Wins

One step out performed the rest. The step compares a 1,092-acre base to a 24,296-acre mapped perimeter after 22.3 times growth on the Little Giant fire in Washington. The field prediction is 23,813 acres, and its growth IoU is 0.814 on that step.

Figure 5. Example predicted perimeter.



8. Improvement Breakdown

The first useful shift was to predict a burned-probability field over a fine grid rather than push a single edge out in radial sectors. The grid detections contribute local evidence in relation to the base, surrounding detections, and the overall detection-cloud shape. It also let the model express which nearby cells looked more like burned ground.

Field version	Change	Reported result or lesson
v1	Per-cell probability model and a top-probability area selector	On the 172-step wide cohort, learned-k reported growth IoU 0.343 and whole IoU 0.680.
v2	Surroundedness at multiple scales, frontier push, cloud anisotropy	Raised the reported oracle shape bound; best real explosive result reported around 0.43 to 0.44.
v3	More neighborhood scales, density ratios, earliest detection time, DEM context	Reported explosive growth IoU up to about 0.45. More terrain did not solve the area cutoff.

Knowing where and how much

The field can rank plausible burned cells very well. The retained exp_hi per-cell AUC is about 0.979 and average precision about 0.747. But selecting the correct quantity of those cells is a different problem. A true-area oracle improves the score, which says the field has shape information; unfortunately, however, none of the tested live selectors could reliably know how much area to paint on the most explosive cases.

Knowing where and how much is specifically important, as demonstrated earlier; while acreage can be decently accurate, the real perimeter IoU can be drastically different. Since direction is extremely important to this tool, it cannot be excluded from any model.

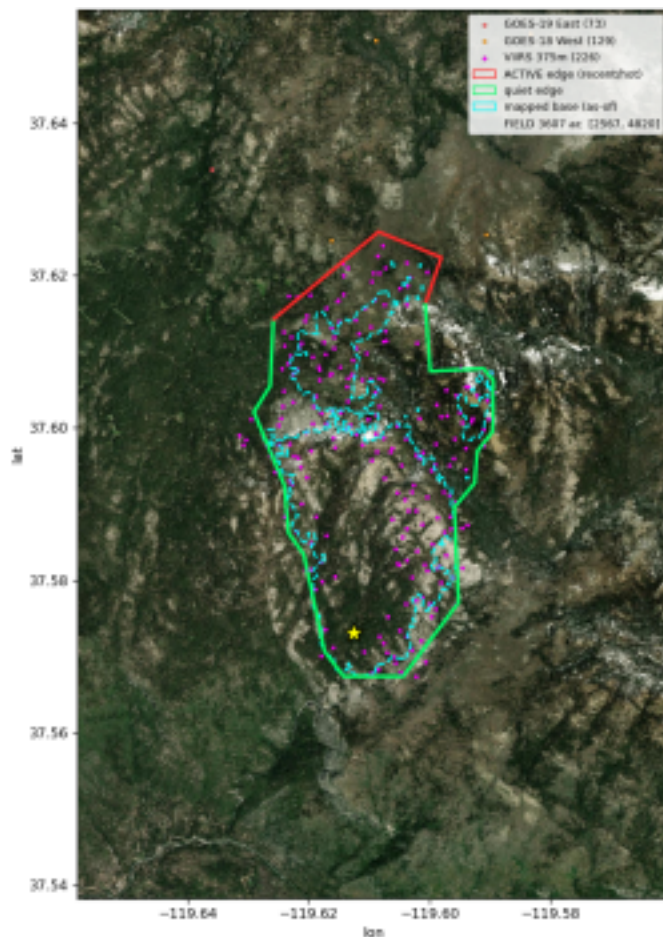


Figure 6. Satellite detections and predicted perimeter.

9. New Plan

At first the explosive-fire miss looked like the perimeter operator was discarding useful high-FRP detections. The purpose of the X1 audit was to credit-check every outside-base detection against both the real growth ring and the prediction. It found that the champion had already counted almost all missed in-growth detections, in other words, meaning that they weren't filtered out. The operator simply placed the edge short of them. That killed a proposed filter-gate fix and pointed to reach and shape instead.

Then after running a deeper analysis I identified a two-sided problem. The champion was too cautious on the largest bursts, but it also added false growth on flat periods. Across positive-growth steps, the fitted response exponent was about 0.5: a ten-times larger real change led to only about a 3.2-times larger prediction. A global relaxation helped some extremes and badly hurt other fires.

Ablating the cap, percentile, and smoothing globally helped several under-reaching examples, then cut growth IoU by 76 to 92 percent on fires where the champion was already close. The per-sector time-slope idea also failed because each sector often lights up and then stays at a similar detection distance. The timing signal is in which sectors light up, not in the slope inside one sector.

Eventual shared explanation

A large share of apparent false positives fall inside a later mapped perimeter of the same fire. The model can see ground that burns later, while the reference perimeter records an earlier snapshot. That makes some errors look spatial when the missing information is the exact burn time. After deeper analysis, I found that 71 percent of explosive-stratum false-positive cells inside a later same-fire perimeter, versus 23 percent for matched cells the model correctly rejected.

10. Failed Experiments

The field produces probabilities. Then, a selector turns those scores into one polygon by choosing how many cells to keep. Since the field ranked cells well, a natural bet was to learn this cutoff. To fix this, I tested multiple ways to estimate the per-step multiplier k .

Selector idea	What it tried	Outcome
Fixed cutoff / fixed k	One operating point across fires	Strong simple baseline; good shape result but poor acreage on the hardest explosive fires.
Learn k from scene features	Fire size, detection count, extent and FRP summaries	Improved over some earlier field settings, but weak transfer on the highest-growth group.
Learn k from prior momentum	Prior-step and pre-window GOES activity	Did not provide a stable held-out cutoff.
Use probability distribution shape	Infer amount of growth from the field score distribution	Did not beat a fixed value in the hardest stratum.
Expected-loU threshold	Pick threshold from the field expected overlap	Matched the tuned constant; did not separate explosive severity.
Per-sector thresholds	Let different directions use different cutoffs	No reliable gain; sometimes hurt growth overlap.
Oracle k	Use truth only to see the possible headroom	Improved the score, proving the cutoff matters, but cannot be deployed.

The reported shape-optimal k on the high explosive group could raise growth loU from about 0.45 to 0.53 and acreage accuracy to about 88 percent (still below my goal). The live choices stayed around 0.42 to 0.46 loU and 0.31 to 0.35 acreage error. That leaves headroom, but the available features did not identify which step needed which cutoff.

The learning-curve analysis also reversed an earlier explanation. More of the same cases did not meaningfully improve k prediction. The evidence pointed to missing information in the inputs and uncertain truth timing, rather than simply too few training fires.

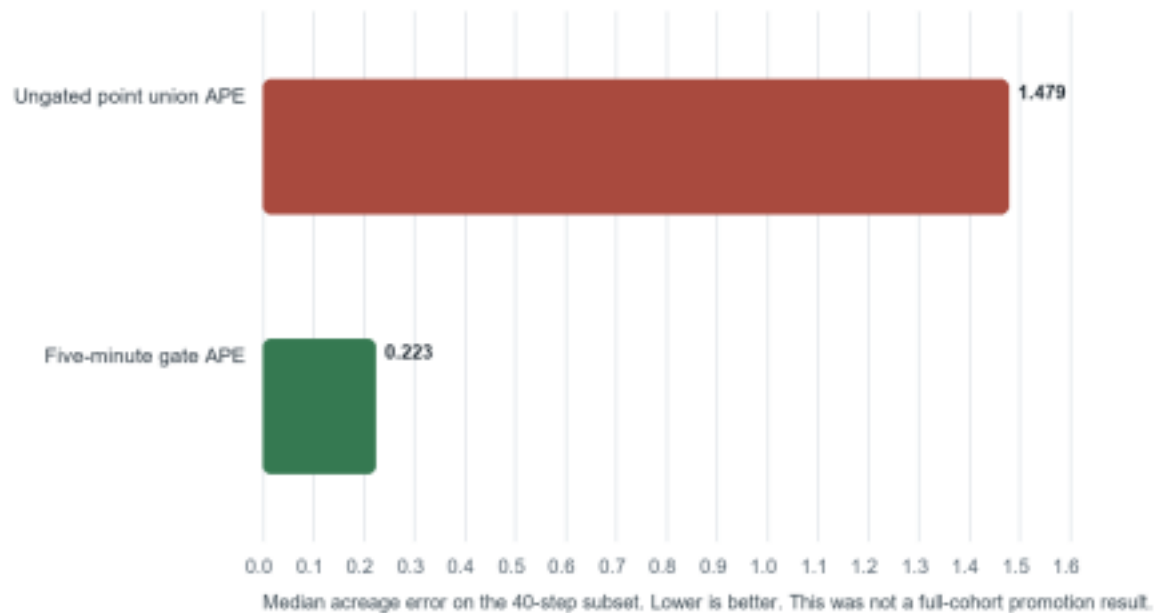
11. Five-minute updates

The five-minute tracker idea was more specific than a periodic task runner: keep an anchored state, advance in five-minute ticks, and admit an observation only if the fire could plausibly have reached it since the last state update. This tested whether time-local reachability could reject detections that are close in space but too fast to have come from the active front. The purpose of the five-minute updates was for situational awareness, but originally it was just something I was really interested in as civilians like myself don't have public or free access to the same resources that fire agencies such as CalFIRE had I wanted to make something myself that could give me that ability.

Test	Sample and method	Result	Decision
Anchored five-minute gate	40-step subset; reject detections outside a 100 m/min reachable envelope	APE 1.479 to 0.223; growth precision 0.083 to 0.244; 64 percent rejected.	Mechanism looked real, but the tracker still lagged the field model.
Gate complementarity	60-step subset; compare true and false field cells to admitted or rejected detections	On only 10 exp_hi steps, true positives were farther from the nearest admitted detection than false positives were from a rejected one.	Interesting, small-sample lead.
Gate features in field model	Add reachability to the burned-probability field and test on held-out fires	No lasting promotion result appears in the retained final notes.	Do not present as a validated improvement.

Full five-minute causal replay	Use raw FDCC granules with acquisition cutoff and exact source manifests	69,284 GOES-18/19 FDCC granules were hash-verified across 759 steps.	Strong data/replay foundation, but data is FDCC masks only and receipt time is absent.
Current state replay	Build a scheduled live loop and score at real mapped timestamps	Prospective capture and fallback paths were implemented.	No five-minute reference truth set exists in the saved evaluation.

The five-minute reachability experiment



12. Raw satellite reconstructions

A separate September 7 audit compared the actual live dual-GOES champion with a legacy single-GOES cache. They were not equivalent data sources. The live version reproduced exactly from its frozen ordered feed on 759 of 759 steps, with zero errors. Yet the live and old-cache detection sets differed on 676 of 759 steps, and the output geometries differed on 638 of 759. Through this, I found that the old cache should not be used as a stand-in for the live dual-satellite system.

On 116 explosive steps, the live champion median growth IoU was 0.213, acreage APE was 0.306, recall was 0.307, and precision was 0.483. Full physical boundary length was about 74.6 percent of the mapped reference length. Length is a shape-size diagnostic, not an overlap score. These results, while very humbling, proved to me that I was going in the wrong direction and I needed to change something up.

Raw FDCC reconstruction methods kept real satellite pixel footprints and separate fire, observed-nonfire, and unavailable states; however, saturated pixels remain unknown. These are good provenance and geometry choices. However, I found the full-cohort evaluation did not promote a raw-feed method. In a 759-step A/B/C audit, LOFO kept the champion in all 83 folds. The 1 km advisory envelope added a small explosive IoU gain but failed other safety gates.

13. Crossed-view and sensor tests

A GOES pixel is spatially broad (2km by 2km) and can be displaced by viewing geometry and slight

calibration errors. One way to get a sharper location is to compare GOES-East and GOES-West footprints where both sensors observe the same fire. Their view directions cross. This geometry carries more localization information than brightness or FRP alone.

Idea	Measured result in saved notes	Takeaway
Crossed footprint geometry	On a 27-step VIIRS comparison, growth IoU was reported to improve from about 0.13 to 0.28. A later proper-powered full-cohort analysis found support was sparse.	Real signal, but sparse coverage. Not enough by itself to guarantee a strong five-minute perimeter.
FRP weighted fill	0.43 to 0.39	Hurt. FRP follows the flaming front, not the whole burned extent.
FRP plus gradient	0.43 to 0.37	Hurt. More intensity emphasis did not recover area.
Shrink by sub-pixel burning area	0.28 to 0.24	Hurt. The reported area is the flame fraction, not the burned region.
Mask grading	About 0.28 to 0.28	Neutral.
Temperature as growth magnitude	Spearman about +0.11	Did not predict how much ground had burned.
Clear-pixel subtraction	Tested against crossed footprint geometry	No retained promotion result; remained a separate diagnostic.

The full 31-step analysis reported crossed geometry covering about 13.9 percent of cells and appearing in 13 steps. This means there is some correlation, but current GOES view overlap does not cover enough of the problem to act like a finer always-on sensor. VIIRS remains much sharper when it passes, but it does not arrive every five minutes, a major downside. Recently, new satellites have shown promising results for monitoring fires (recent social media post from CalFIRE). If data is publicly available, it would be very helpful towards tuning this project.

14. Other architectures and their stop signs

Attempt	What it asked	What stopped it
Arrival-time surface	Can one monotone burn-arrival map explain the detections and mapped snapshots?	Forward-only arrival model reached exp_hi growth IoU around 0.156 versus about 0.45 for the field. Held-out arrival timing was roughly 600 minutes wide on a 24-hour window.
Physics-constrained assimilation	Can Rothermel timing identify which detections belong to the active fire?	Time-normalized reachability had AUC 0.500 overall and 0.429 on exp_hi. Plain distance did better.
U-Net spatial model	Can a CNN improve cell ranking beyond the tree model?	Worse AP on exp_hi. Rank blending was statistically indistinguishable from the per-cell model.
Contour perimeter extraction	Can marching squares improve shape compared with selected grid cells?	A wash after matching polygon area.
Global operator relaxation	Can removing the cap, smoothing and high percentile fix underreach?	Helped some fires, badly harmed others already captured by the old operator.
Persistence as reach gate	Can repeatedly observed detections be trusted further from the edge?	Persistence was common in both real fronts and contamination. It did not separate reach.

15. Learned Field V3

The current best model is named learned_field_v3. It builds on previous models. Fundamentally, the model only sees the last mapped perimeter and GOES and VIIRS detections up to a cutoff time, like all

other models. Each cell outside the base perimeter has 55 features falling under distance, neighboring cells, surroundedness, detection timing, density ratios, frontier push, cloud anisotropy, fire-wide context, and terrain. To solve earlier issues of poor learning, a gradient-boosted tree classifier was trained on past fires. In this case, burned cells are compared to unburned cells, and probabilities are corrected. The selector sums every cell's probability to estimate how many cells will burn, multiplies by k, and keeps that many of the most likely cells. It then uses union with the base perimeter. Everything is validated leave-one-fire-out, so that the model is always scored on a fire it never trained on.

From v1 to v3 of the learned field models, true improvement was made. The first model received a growth IoU of 0.343 and a whole IoU of 0.680 on the whole cohort. v1, compared to other models, added the per-cell probability selector. After this, surroundedness, frontier push, and cloud anisotropy were added and improved the explosive result IoU to 0.43. Finally, v3 implemented more neighborhood scales, density ratios, detection times, and DEM context boosting, specifically the explosive growth step IoU, to about 0.45 with much greater results such as 0.84 IoU on a 22x step in the Little Giant Fire (Washington). As a whole, the v3 model improves on the radial champion, as the X1 audit showed it did not learn how to properly place the edge of the fire and disregarded many real GOES values. The improvement on this in v3 was that, instead of flatly growing the fire, probability was added to each cell to identify patterns.

Another thing v3 improved on was suppressing scatter. Scatter was a key issue, as detections carry lots of noise, with a median offset of about 1.7km from where the fire really is. This, in combination with the already large GOES tile size, created many odd-looking fire perimeters. The v3 learned field uses surroundedness, density, and timing features to learn which detections mark real spread and which are noise. This replaces hard gates that merged neighboring fires or severely cut off/undestimated fire growth.

Explosive steps, the key limitation in previous models, also significantly improved. The former "Champion" model had a growth IoU on explosive steps >3x of 0.21 and an acreage accuracy of 52%. The new v3 model received a score of 0.45 and 68% average acreage accuracy. Across every step, the learned field v3 reached 85.8% acreage accuracy and 0.70 whole perimeter IoU, marking significant gains. This improvement likely came from the improvement in the quality of rankings. v3 learned field with a per-cell ranking reached an impressive AUC of about 0.979 and an average precision of 0.747.

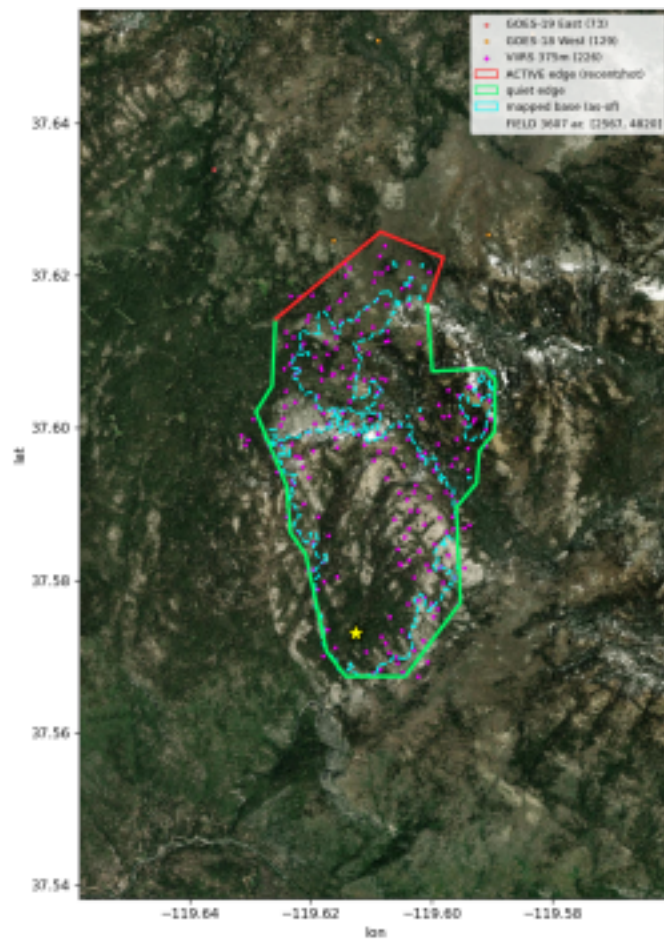
Clearly, however, v3 is not without its downsides. The major issues with v3 still lie on how much area to consume, terrain, and timing. For terrain specifically, adding terrain provided no real benefits to any precision, so it was left out.

To improve on v3, weather must be reincluded. Previously, it was excluded due to poor data; especially when determining real wind speed and fire growth, it would consistently provide poor growth direction. Additionally 21. Contamination's plan to reconfirm explosive steps should filter out any fires with absent or missing data. Finally, all data should be included when available, such as cameras, aircraft, additional satellite data, etc. These issues should be addressed in v4.

Results

While an IoU of 0.45 on explosive steps shows improvement, it is not good enough to ship. However, running the learned field v3 on the dome fire (live) produced a promising result, with it being able to identify an active fire edge and a fire perimeter representative of the ground truth.

Additionally, the learned field v3 showed promising results on extremely explosive steps with accurate data. One key example was the Little Giant Fire's 22x step, which produced an IoU of 0.814 and an extremely accurate perimeter. This suggests that an issue may be missing data within explosive steps, leading to inaccuracies in the perimeter shape.



16. Timeline

Date or period	Change	Why it happened / result
May 28, 2026	The nested fpm Git history starts with the smoke-detection project.	A camera-based alert path was the first saved code lineage.
May 31	Fire spread prediction was added and smoke detection upgraded.	The camera model and perimeter work began as parallel tools.
June 14	Simulation fixes covered hotspot parsing, nonburnable zones and a 72-hour cap.	The physics side became more usable, but remained distinct from the later perimeter model.
June 24	The root repository begins with a GOES-19 fire monitoring map.	The monitoring map became the visible starting point for the satellite work.
June to July	The camera review workflow, dataset export, model versions and live GOES replays were built out.	Iterative annotation and monitoring supported both tracks. Root Git records little of this detail; the local fpm history and logs retain more.
Late July	GOES replay code and production tooling were revised.	Live feeds, replay views and smoke model plumbing were made operational.
September 6 to 7	Explosive underreach, eligibility, gate and operator-oracle audits.	The investigations killed several filter and routing ideas and measured limits of push versus fill.
September 8	Full GOES-18/19 FDCC cohort snapshotted and hashed; Systems A, B, C were audited.	Reproducibility improved. No raw-feed candidate was promoted.
September 9	Learned burned-probability field and v1 to v3 shape work.	Improved explosive growth overlap, then selector and truth-timing limits became the focus.

September 10	Arrival-time, physics assimilation, reachability and image-model studies were summarized.	Several architectures were refuted; the anchored 5-minute gate showed a small diagnostic win.
September 15 to 18	Dome live fusion artifacts and final result figures/docs were saved.	Good examples were preserved, while the research status still called for future causal validation.
September 27 review	This report reconciles the retained documents and labels cohort differences.	No newer dated empirical report was found in the workspace.

17. Full named perimeter experiment register

The numbered exp scripts are retained as named experiments. This index gives the question each asked and the broad conclusion supported by the saved result notes. Several were exploratory and do not have a separately reported final metric; those are marked as diagnostic rather than quietly counted as wins.

Experiment script	Question or change	Disposition from retained notes
exp1.py	Change operator constants, rescale magnitude and try direct shape families.	Exploratory first sweep; motivated oracle and magnitude checks.
exp2_oracle.py	Measure oracle headroom for shape families.	Diagnostic ceiling; showed shape quality and area selection must be separated.

exp3_magnitude.py	Compare observables as predictors of growth area.	Leave-one-fire-out magnitude investigation; no intensity-only fix.
exp4_ceiling.py	Estimate shape ceiling from detection evidence.	Diagnostic; exposed a high ceiling only when the correct area is supplied.
exp5_cellclf.py	Classify burned cells directly.	Led toward per-cell probability field.
exp6_sector_oracle.py	Measure the best possible per-sector oracle.	Showed limits of radial-push shape vocabulary.
exp7_sectorreg.py	Learn per-sector reach with held-out fires.	Did not displace the later field approach.
exp8_quantile.py	Use quantile sector reach and blend with push.	Did not become the best retained method.
exp9_audit_ceiling.py	Audit whether only 52 percent of growth was observable.	Original claim was retracted after a measurement bug.
exp9b_audit.py	Recheck coverage with grid sampling and a KD tree.	Corrected audit found the earlier ceiling claim was wrong.
exp10_field.py	Build a fine-grid burned-probability field.	Major direction change; useful local shape signal.
exp10b_field.py	Vectorize the field and add a subpixel direction prior.	Engineering and shape diagnostic.
exp11_tune.py	Retune field thresholds and combine with frontier.	Showed shape gains could come with overpainting.
exp12_calib.py	Calibrate field magnitude from probabilities.	TOPK area selection became the working interface.
exp13_iso.py	Isotonic calibration and size-aware TOPK.	Tested probability calibration and fire size.
exp14_oraclek.py	Use a truth-aware area oracle to test cutoff headroom.	Confirmed the selector could matter substantially. Oracle only.
exp15_kpred.py	Predict per-step k with LOFO.	Some overall gains; weak on highest explosive group.
exp16_field2.py	Add surroundedness, frontier push and cloud anisotropy.	Field v2 improved real and oracle shape results.

exp17_field3.py	Add multiscale neighborhoods, density and terrain.	Field v3 reached about 0.45 exp_hi growth IoU; area choice remained hard.
exp18_prewindow.py	Use pre-window GOES momentum for k.	No reliable gain beyond noise.
exp19_learningcurve.py	Test whether more fires fix the k predictor.	Learning curve was flat; data-starvation explanation was withdrawn.

Experiment script	Question or change	Disposition from retained notes
exp20_where.py	Locate exactly where exp_hi growth was missed.	Most missed growth remained rankable; little was fully invisible.
exp21_selfk.py	Infer k from probability-field shape and optimize for shape.	Oracle shape cutoff reached about 0.53; field-derived k did not match it.
exp22_expiou.py	Choose cutoff by expected IoU without truth.	Matched a tuned constant; did not discriminate severity.
exp23_sectorcut.py	Use direction-specific thresholds.	No reliable gain; sometimes worse.
exp24_contour.py	Extract a contour instead of cell squares.	Wash after matching predicted area.
exp25_tick.py	Build an anchored 5-minute tracker with physical reachability.	APE improved in the 40-step diagnostic; not a replacement model.

exp26_gatecomp.py	Check if reachability adds information beyond the field.	Promising but small n, especially only 10 exp_hi steps.
exp27_gatefeat.py	Add reachability features into the field.	No promotion result in final retained records.
exp28_truthstrata.py	Test accuracy against truth provenance and label timing.	Supported label timing as a real limit.
exp29_crossed.py	Test intersection of opposing GOES footprints.	Early feasibility signal against VIIRS.
exp30_accum.py	Accumulate crossed footprint intersections over time.	Tested as a perimeter method; not sufficient on its own.
exp31_crossfeat.py	Use crossed geometry as per-pixel model features.	Coverage remained sparse in later full analysis.
exp32_clearsub.py	Subtract clear pixels to sharpen crossed geometry.	Diagnostic, no promotion claim.
exp33_crossed_full.py	Repeat crossed-view test with exact native footprints.	Confirmed a real geometry signal, but only partial coverage.
exp34_subpixel.py	Test area, temperature and power subpixel retrieval.	Subpixel fire area did not identify burned extent.
exp35_frpweight.py	Weight crossed footprints by FRP.	Hurt relative to geometry alone.
exp36_frpcontrast.py	Test local FRP contrast.	Intensity contrast did not reliably distinguish true burned cells.
exp37_terrainfuel.py	Test terrain and fuel in the sensor discrimination framework.	Useful physical context, but no evidence it removes the main timing and magnitude limit.

Separate September 6 to 7 operator diagnostics

These X-series tests are not the exp1 to exp37 scripts. X1 audited whether the champion threw away useful detections. X2 compared push and fill on a tiny-base sample and found fill had higher recall but poor area control. X3 repeated that comparison on 115 of 116 explosive steps. An oracle could choose the better arm per step, but cloud/base extent, detection count and base acreage did not produce a usable

LOFO selector unfortunately. This meant that push remained the general point estimator; fill stayed diagnostic.

18. What the numbers mean

What is supported

- The smoke mask model has a meaningful, grouped validation result around 0.79 mAP50 in the saved summary.
- The split correction removed a known camera-frame leakage problem.
- The burned-probability field materially improved explosive growth overlap on retrospective, leave-one-fire-out development cohorts.
- Dual-GOES frozen-feed replays and raw FDCC manifests provide a substantial reproducibility foundation.
- An anchored reachability gate can reduce impossible growth in a small five-minute diagnostic.
- Some camera bearings, satellite geometry and VIIRS points can add useful local context.

What is not supported

- A 0.75 growth-loU perimeter model.
- Five-minute spatial accuracy, because no five-minute perimeter truth set is present.
- Historical live availability from acquisition time alone.
- The claim that model score on a single camera image equals field detection accuracy.
- The claim that a good single fire picture is representative of all fires.
- A camera smoke-to-perimeter improvement on a held-out cohort.
- Production promotion of raw reconstruction, VIIRS boundary update, or the candidate envelopes.

A clean next evaluation would archive the estimate before the outcome is known, record source receipt time and coverage, compare only against truth with a defensible acquisition timestamp, keep fires separated between fitting and evaluation, and report both growth overlap and acreage/precision safety checks. The prospective capture harness and frozen replay tooling are useful starts.

21. Contamination

Fire Step Data Validation Plan

The poor results of the model in times of excessive growth could point to an error where data is either corrupted (missing a timestamp) or missing in explosive fires. Since the algorithm works with knowledge of the fire's comprehensive FRP history, missing FRP values for certain tiles would lead to gross underestimations. In order to combat this I would need to go through every single explosive step and look at all the data available for each step OR create/tune a validation system that validates every step again.

21. Next Steps

Camera Project

- Label and ingest HPWREN data

- Reach 7000+ images in training dataset and solve label issues
- Run system 24/7 on AlertWEST/AlertCalifornia 360 degree cameras and test speed and reliability

Wildfire Project

- Reach 0.70+ median IoU (with fire step data validation plan) on explosive fire steps -
Reach 90% acreage accuracy on explosive fire steps
- Implement radio transcriptions to tune perimeter system
- Implement aircraft tracker and drop detector to constrain fire perimeters